

Script de présentation et de tournage — DVWA

Master SSI • Étude des attaques web et contre-mesures

Version du document : 1 • 12 septembre 2026

Présentateur : [ton nom] • Établissement : [à compléter] • Encadrant : [à compléter]

Source étudiée : DVWA b496a5d3de6b967410155e1b7d3e51e9d035eb22.

Comment utiliser ce document

Les passages « À dire » sont destinés à être prononcés. Les passages « À filmer » sont des consignes de manipulation. Les résultats annoncés sont des attentes issues de la lecture du code, pas des observations de ta vidéo. Répéter chaque scénario avant l'enregistrement et ne prononcer une conclusion positive qu'après avoir vu sa preuve.

Ce document couvre les 19 modules et leurs principaux sous-exercices. Il ne prétend pas énumérer toutes les variantes d'exploitation possibles. Les niveaux medium et high doivent être examinés avec leurs sources ; ils ne sont pas systématiquement des corrections du niveau précédent.

État réel : les deux sites démarrent, les 38 pages de modules et les routes API de santé ont été contrôlées. Les attaques navigateur, les corrections personnelles et leurs rejouements ne sont pas encore tous validés.

La copie sur le port 4281 est une copie de travail encore vulnérable. Ne jamais la présenter comme sécurisée aujourd'hui.

Format de la vidéo

Prévoir environ 70 à 100 minutes pour une démonstration complète avec manipulations ; le temps dépend des répétitions, de la cryptographie et des échanges HTTP. Découper en chapitres ou en plusieurs vidéos si la durée imposée est plus courte.

Plan : introduction et architecture (5 min), injections (10 min), XSS et navigateur (12 min), fichiers et accès (15 min), sessions et CAPTCHA (8 min), CSP et JavaScript (8 min), cryptographie (10–15 min), API (10–15 min), bilan (5 min). Ce minutage est indicatif.

Pour une soutenance courte : présenter un résumé de 15–20 minutes et fournir les 19 séquences complètes en annexes vidéo. Ne pas accélérer au point de masquer les preuves.

Préparation de l'enregistrement

1. Ouvrir <http://127.0.0.1:4280> et <http://127.0.0.1:4281> dans deux profils de navigateur distincts. Les ports ne séparent pas les cookies de niveau de sécurité DVWA.
2. Utiliser uniquement les comptes et données fictives DVWA. Compte initial admin / password. Pour les accès non administrateurs : gordonb / abc123, à confirmer avant tournage.
3. Dans DVWA Security, régler low et montrer le niveau avant chaque essai. Ne pas supposer que le port 4281 implique impossible.
4. Préparer Burp Suite : Proxy → HTTP history → sélectionner la requête → Send to Repeater. Activer l'interception uniquement quand nécessaire, puis la désactiver pour laisser naviguer le site. Un 302 vers login indique une session manquante, pas une protection de la faille.
5. Filmer navigateur, requêtes et réponses avec une police lisible ; couper les notifications et fermer les onglets personnels. Faire une prise de son de vingt secondes et la réécouter.
6. Préparer les fichiers témoins du dossier docs/video/fixtures. Ne pas téléverser de fichiers personnels. Ne pas utiliser de webshell : le témoin PHP fourni affiche seulement un texte fixe.
7. Pour chaque scénario, sauvegarder requête, réponse, effet et capture. Nommage : 01-sqli-low-requete.txt, 01-sqli-low-effet.png, etc. Noter compte, niveau, date et état initial.
8. Les essais CSRF et CAPTCHA changent un mot de passe. Noter sa nouvelle valeur, puis le restaurer par le formulaire légitime. Ne pas réinitialiser toutes les bases sans avoir sauvegardé les preuves.
9. Le serveur de page témoin proposé plus loin n'est pas lancé automatiquement. Les dépendances externes CAPTCHA ne sont pas configurées. Les scènes correspondantes ont une variante « limite rencontrée ».
10. Ne pas rejouer toutes les attaques avant d'enregistrer sans remise en état : XSS stockée, rôles API modifiés, verrouillages et cookies BAC peuvent fausser les résultats suivants.

Introduction — texte à dire

« Bonjour, je suis [nom], étudiant en Master Sécurité des Systèmes d'Information. Ce projet porte sur l'étude expérimentale des attaques web et de leurs contre-mesures.

Mon objectif est d'expliquer comment une entrée contrôlée par un utilisateur devient une requête SQL, du code exécuté, une action non autorisée ou une donnée divulguée. Je vérifie ensuite quelles protections empêchent cet effet et si le fonctionnement légitime reste possible.

J'utilise DVWA, une application PHP et MariaDB volontairement vulnérable. Je n'ai pas développé DVWA : mon travail consiste à préparer le laboratoire, analyser ses sources, construire les tests, étudier les protections existantes et, dans une phase distincte, développer et valider mes propres correctifs.

Les essais se déroulent exclusivement sur mes instances locales. Les comptes et les données sont fictifs. Une plateforme pédagogique rend les mécanismes observables, mais ne reproduit pas toute la complexité d'un site de production. »

Architecture — à filmer et à dire

À filmer : compose.yaml, ports, deux instances, VERSION. Éviter de faire défiler tout le fichier.

« Le navigateur rejoint un proxy Nginx sur les ports locaux 4280 et 4281. Chaque DVWA possède sa propre base MariaDB et son réseau interne. Les bases ne sont pas publiées sur la machine. Les données sont persistantes.

Le premier site sert de référence. Le second est destiné à nos modifications ; à ce stade, il reste vulnérable. Les sources et les images sont identifiées pour pouvoir reproduire l'expérience.

Je couvre les dix-neuf modules de cette version, y compris le contrôle d'accès BAC qui n'apparaît pas dans le menu principal. »

Méthode commune — à dire

« Pour chaque cas, je montre d'abord un usage normal. Je modifie ensuite une entrée en formulant une hypothèse précise. J'observe l'effet, puis je relie cet effet à la ligne de code concernée. Enfin, je compare avec la protection existante ou notre correctif, et je rejoue un usage autorisé.

Un code HTTP 200 n'est pas une preuve de réussite. De même, une erreur HTTP ne suffit pas à prouver une protection. La preuve porte sur la donnée divulguée, le script exécuté, la modification persistante ou la décision d'autorisation. »

01. Injection SQL classique

Module : /vulnerabilities/sqli/ • Sources : upstream/vulnerabilities/sqli/source/

À dire avant le test

« Le champ attend un identifiant d'utilisateur. Au niveau low, sa valeur est insérée dans une chaîne SQL. Je cherche à modifier la condition de sélection. »

À filmer et à faire

1. Soumettre 1 : filmer le nom retourné.
2. Dans User ID, saisir exactement :
`1' OR '1'='1' #`
3. Filmer les différents utilisateurs retournés. Dans un champ de formulaire, # est envoyé correctement ; dans une URL écrite à la main, il doit être encodé en %23.
4. Ouvrir View Source, low.php : repérer l'interpolation de \$id dans SELECT.
5. Sur l'autre profil, sélectionner impossible ; envoyer la même valeur par le formulaire, avec son jeton valide. Puis rejouer 1.

À dire si l'effet est observé

« La réponse contient plusieurs utilisateurs alors que l'usage normal sélectionnait un identifiant. L'entrée a changé la logique de la requête SQL. »

Comparaison et contre-mesures — à dire

« La version impossible utilise notamment un contrôle numérique et une requête préparée. Pour attribuer le résultat à la paramétrisation, j'examine le code : ce seul essai pourrait déjà être arrêté par la validation numérique. Notre futur correctif sera testé séparément. »

Limites et remise en état

Ne pas réutiliser la conclusion de notre ancien portail : dans DVWA, ce scénario démontre une extension de la sélection, pas un filtre de propriétaire Alice/Bob.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

02. Injection SQL aveugle

Module : /vulnerabilities/sqli_blind/ • Sources : upstream/vulnerabilities/sqli_blind/source/

À dire avant le test

« Une injection peut exister même si le serveur ne retourne pas les lignes de la base. Je compare ici les réponses à deux conditions dont je connais la valeur. »

À filmer et à faire

1. Soumettre 1 et un identifiant absent, puis observer les différences.
2. Envoyer successivement :
`1' AND '1'='1' #`
`1' AND '1'='2' #`
3. Comparer le message d'existence et, dans Repeater, statut et taille de réponse.
4. Au niveau impossible, refaire l'essai avec un formulaire et un jeton frais ; tester ensuite un identifiant normal.
5. Pour un complément temporel, prévoir une séquence séparée avec contrôles et répétitions ; ne pas conclure à partir d'un seul délai.

À dire si l'effet est observé

« La réponse distingue une condition vraie d'une condition fausse injectées dans la requête. Cette différence fournit un canal d'information même sans afficher les données. »

Comparaison et contre-mesures — à dire

« La séparation entre requête et paramètres reste la protection principale. Masquer les erreurs ne supprime pas une injection. Une analyse temporelle exige de distinguer un délai provoqué du bruit de réseau. »

Limites et remise en état

Une paire booléenne suffit à cette scène ; elle ne constitue pas une extraction complète de la base.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

03. Injection de commandes système

Module : /vulnerabilities/exec/ • Sources : upstream/vulnerabilities/exec/source/

À dire avant le test

« Le service effectue un ping. Je vérifie si la saisie est traitée comme une adresse ou interprétée par un shell. »

À filmer et à faire

1. Au niveau low, soumettre 127.0.0.1 et attendre le résultat du ping.
2. Soumettre :
127.0.0.1; printf VIDEO_COMMAND_OK
3. Filmer le marqueur dans la sortie. La commande supplémentaire ne lit aucun secret et ne modifie aucun fichier.
4. Comparer low.php et impossible.php. Rejouer la saisie puis l'adresse normale au niveau impossible.

À dire si l'effet est observé

« Le marqueur ne provient pas du ping : il est produit par la commande ajoutée à ma saisie. Cela prouve une exécution supplémentaire dans le conteneur. »

Comparaison et contre-mesures — à dire

« Les listes de caractères interdits des niveaux intermédiaires demandent une analyse de contournement. Le niveau impossible reconstruit une adresse numérique. Pour notre correctif, nous privilégierons une validation d'adresse et un appel sans shell lorsque possible. »

Limites et remise en état

Le résultat est une exécution dans le conteneur DVWA ; il ne prouve pas une compromission de la machine hôte.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

04. XSS réfléchi

Module : /vulnerabilities/xss_r/ • Sources : upstream/vulnerabilities/xss_r/source/

À dire avant le test

« Le site réaffiche un nom fourni dans la requête. Je vérifie si cette donnée devient du code JavaScript dans la réponse. »

À filmer et à faire

1. Soumettre Bonjour et montrer son affichage normal.
2. Soumettre :
`<script>alert('XSS_REFLECHIE')</script>`
3. Filmer la boîte de dialogue et la requête correspondante. Fermer la boîte.
4. Au niveau impossible, soumettre le même texte, puis un nom normal.
5. Montrer htmlspecialchars dans la source de la protection.

À dire si l'effet est observé

« Le navigateur exécute le script fourni dans le nom. Cette XSS est réfléchi : la charge arrive dans la requête qui produit la réponse. »

Comparaison et contre-mesures — à dire

« L'encodage adapté au contexte empêche de traiter le nom comme une balise. Les filtres de chaînes des niveaux intermédiaires ne remplacent pas cet encodage. »

Limites et remise en état

L'exécution est la preuve ici. Aucune lecture de cookie ni transmission externe n'est nécessaire.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

05. XSS stockée

Module : /vulnerabilities/xss_s/ • Sources : upstream/vulnerabilities/xss_s/source/

À dire avant le test

« Le commentaire est conservé en base puis affiché à d'autres visiteurs. Je teste donc une exécution différée dans une autre session. »

À filmer et à faire

1. Publier un message normal avec le nom Video.

2. Dans Message, publier :

```
<script>alert('XSS_STOCKEE')</script>
```

3. Ouvrir le même module et la même instance dans un profil victime distinct, connecté et réglé sur low. Filmer le déclenchement sans nouvelle publication.

4. Sur la seconde instance au niveau impossible, publier la charge puis visiter depuis une autre session ; publier aussi un commentaire normal.

À dire si l'effet est observé

« Le script s'exécute lors de la consultation d'un message déjà enregistré. Le visiteur n'a pas eu besoin de saisir la charge : la base a conservé le contenu dangereux. »

Comparaison et contre-mesures — à dire

« La protection doit être vérifiée pour chaque champ et chaque contexte d'affichage. Je compare ici la gestion du nom et du message dans les sources. Une CSP peut compléter les corrections sans remplacer le traitement du contenu. »

Limites et remise en état

La charge peut se redéclencher à chaque visite. Réserver un état de base à cette scène et réinitialiser seulement après sauvegarde des preuves. Les deux instances ne partagent pas leur base.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

06. XSS basée sur le DOM

Module : /vulnerabilities/xss_d/ • Sources : upstream/vulnerabilities/xss_d/source/

À dire avant le test

« Cette fois, je suis le chemin entre une valeur de l'URL et une écriture HTML effectuée par JavaScript dans le navigateur. »

À filmer et à faire

1. Choisir French normalement.
2. Sur low, ouvrir l'URL locale fournie dans fixtures/dom-xss-url.txt. Elle place la charge suivante, encodée, dans default :

```
</option></select><script>alert('XSS_DOM')</script>
```

3. Filmer l'exécution et, dans les sources, document.location.href puis document.write.
4. Comparer le DOM final à la réponse HTTP. Au niveau impossible, rejouer le cas et choisir de nouveau French.

À dire si l'effet est observé

« Le JavaScript de la page transforme une partie de l'URL en HTML actif. L'observation du DOM complète donc l'analyse de la réponse du serveur. »

Comparaison et contre-mesures — à dire

« Il faut construire le DOM avec des opérations sûres et contraindre les valeurs attendues. Dans cette version, j'examine également la différence de décodage d'URL entre niveaux, sans présenter ce mécanisme seul comme une stratégie universelle. »

Limites et remise en état

Le comportement dépend du contexte HTML et du décodage du navigateur. Répéter cette scène avant tournage ; une charge simplement affichée ne prouve pas une exécution.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

07. CSRF : modification non consentie

Module : /vulnerabilities/csrf/ • Sources : upstream/vulnerabilities/csrf/source/

À dire avant le test

« Le navigateur joint une session à une requête provoquée depuis une autre origine. Je teste si le serveur distingue l'intention de l'utilisateur d'une navigation déclenchée ailleurs. »

À filmer et à faire

1. Se connecter sur 4280, niveau low. Noter le mot de passe actuel.
2. Dans un terminal séparé, lancer :

```
cd /home/modou/dvwa-study  
python3 -m http.server 4290 --bind 127.0.0.1 --directory docs/video/fixtures
```
3. Ouvrir <http://127.0.0.1:4290/csrf.html> dans le même profil victime, puis cliquer le bouton Référence.
4. Observer la réponse. Se déconnecter puis vérifier une connexion avec Video-Lab-2026!. Restaurer password par le parcours normal.
5. Sur 4281, sélectionner impossible et utiliser le bouton Copie ; vérifier que le mot de passe initial reste utilisable.

À dire si l'effet est observé

« La requête a été déclenchée depuis une page d'une autre origine et a modifié le mot de passe de la session victime. La reconnexion avec la nouvelle valeur confirme la modification. »

Comparaison et contre-mesures — à dire

« Le niveau impossible vérifie un jeton anti-CSRF et le mot de passe actuel. Pour isoler les protections, il faut ensuite tester chacune séparément. Un futur correctif évitera aussi la modification d'état par GET. »

Limites et remise en état

Les ports 4280 et 4290 créent des origines différentes mais restent same-site. Cette scène ne mesure pas SameSite cross-site. Ne pas présenter un collage direct de la requête dans Repeater comme une preuve navigateur de CSRF.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

08. Inclusion de fichiers et traversée

Module : /vulnerabilities/fi/ • Sources : upstream/vulnerabilities/fi/source/

À dire avant le test

« La page choisit un fichier à inclure d'après un paramètre. Je vérifie si le serveur limite ce choix aux documents prévus. »

À filmer et à faire

1. Ouvrir /vulnerabilities/fi/?page=include.php et les liens légitimes.
2. Au niveau low, ouvrir :
`http://127.0.0.1:4280/vulnerabilities/fi/?page=../../robots.txt`
3. Filmer le contenu du fichier public robots.txt, situé hors du dossier du module ; il sert de témoin sans exposer de configuration.
4. Au niveau impossible, envoyer le même chemin, puis utiliser un fichier légitime.
5. Inclusion distante : réserver une séquence supplémentaire après préparation d'un serveur témoin joignable depuis le conteneur. Ne pas utiliser 127.0.0.1:4290 en supposant qu'il désigne la machine hôte depuis DVWA.

À dire si l'effet est observé

« La traversée du chemin permet de faire inclure un fichier extérieur à la liste fonctionnelle prévue. Le fichier témoin confirme quel contenu a été lu. »

Comparaison et contre-mesures — à dire

« La version impossible limite les noms de fichiers autorisés. Une application générale doit associer des identifiants aux fichiers, contrôler les autorisations et empêcher les chemins de sortir du stockage prévu. »

Limites et remise en état

Lire robots.txt démontre un choix de chemin indu, pas une fuite de secret. L'inclusion distante reste à préparer dans l'architecture isolée ; son absence ne sera pas annoncée comme une protection applicative.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

09. Téléversement de fichier non sûr

Module : /vulnerabilities/upload/ • Sources : upstream/vulnerabilities/upload/source/

À dire avant le test

« Je vérifie si un fichier fourni par un utilisateur peut être placé dans un emplacement où le serveur l'interprète comme du code. »

À filmer et à faire

1. Téléverser le fichier image valide fourni, video.png, et montrer l'usage normal.
2. Au niveau low, téléverser temoin.php depuis fixtures.
3. Ouvrir le chemin retourné par DVWA. Le fichier contient seulement :

```
<?php echo 'VIDEO_UPLOAD_EXECUTED'; ?>
```

4. Filmer le texte produit. Comparer au niveau impossible : tentative PHP, puis image valide.

À dire si l'effet est observé

« Le serveur interprète le fichier PHP téléversé et retourne son résultat. La preuve est l'exécution du témoin, et non le simple succès du téléversement. »

Comparaison et contre-mesures — à dire

« La défense combine le contrôle réel du contenu, les formats autorisés, des noms générés et un stockage empêchant l'exécution. Dans DVWA, je vérifie les traitements exacts du niveau impossible et le bon fonctionnement d'une image. »

Limites et remise en état

Ne pas remplacer le témoin par une console de commandes. Le nom de sortie de l'image peut changer au niveau impossible : suivre le chemin effectivement retourné.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

10. Contournement des autorisations

Module : /vulnerabilities/authbypass/ • Sources : upstream/vulnerabilities/authbypass/source/

À dire avant le test

« Une fonction administrative peut être cachée dans un menu sans être protégée sur ses routes. Je vais comparer un administrateur et un compte ordinaire. »

À filmer et à faire

1. En admin, montrer la page Authorisation Bypass et la requête `get_user_data.php`.
2. Dans un autre profil connecté comme `gordonb`, niveau `low`, constater l'absence du menu puis ouvrir directement `/vulnerabilities/authbypass/`.
3. Tester aussi `/vulnerabilities/authbypass/get_user_data.php` et filmer les données effectivement retournées.
4. Répéter au niveau impossible avec le compte ordinaire, puis vérifier qu'admin conserve son accès légitime.

À dire si l'effet est observé

« Le compte ordinaire accède directement à une fonction ou à ses données malgré l'absence du lien dans son interface. Le menu n'était donc pas une frontière d'autorisation. »

Comparaison et contre-mesures — à dire

« Le contrôle doit être exécuté côté serveur sur la page, les routes de lecture et les routes de modification. Un refus sur la page principale ne démontre pas que ses endpoints sont protégés. »

Limites et remise en état

Si seule une page vide apparaît, ne pas conclure : inspecter les requêtes de données. Ne pas confondre compte DVWA et utilisateurs fictifs du module API.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

11. Broken Access Control : confiance dans un cookie

Module : /vulnerabilities/bac/ • Sources : upstream/vulnerabilities/bac/source/

À dire avant le test

« Le module BAC est présent dans les sources mais caché dans le menu. Au niveau low, une partie de la décision utilise un identifiant de cookie contrôlable par le navigateur. »

À filmer et à faire

1. Se connecter comme gordonb et ouvrir /vulnerabilities/bac/.
2. Dans Application → Cookies, créer ou modifier user_id avec la valeur 2 et chemin / ; consulter /vulnerabilities/bac/?action=view&user_id=2.
3. Modifier user_id en 1 et consulter /vulnerabilities/bac/?action=view&user_id=1.
4. Filmer le profil retourné et le compte connecté. Comparer le code low.php.
5. Au niveau impossible, même compte ordinaire, rejouer le cookie falsifié et la demande du profil 1. Vérifier aussi l'accès au profil 2.

À dire si l'effet est observé

« Le serveur accepte une identité d'objet choisie dans un cookie pour accorder l'accès au profil demandé. La décision est influencée par une donnée fournie par le client. »

Comparaison et contre-mesures — à dire

« La référence impossible retrouve l'utilisateur à partir de la session serveur puis vérifie l'objet demandé. Pour cette scène, le compte ordinaire est indispensable ; un compte autorisé ne permet pas de prouver un contournement. »

Limites et remise en état

Supprimer les cookies user_id et user_role de démonstration après la scène. Le README BAC ne correspond pas exactement à toutes les branches : commenter le code observé, pas seulement sa description.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

12. Force brute et limitation des tentatives

Module : /vulnerabilities/brute/ • Sources : upstream/vulnerabilities/brute/source/

À dire avant le test

« Je teste la possibilité d'enchaîner des essais de mot de passe et j'observe les mécanismes de ralentissement ou de verrouillage. »

À filmer et à faire

1. Dans le module Brute Force low, essayer admin avec faux1, faux2 puis password. Trois essais suffisent à la démonstration de base.
2. Capturer les trois réponses dans l'historique HTTP ; relever les durées sans les transformer en benchmark.
3. Lire medium, high et impossible : distinguer délais, jetons et état de verrouillage.
4. Si l'on filme le verrouillage impossible, relever le seuil et la durée dans sa source avant la prise. Obtenir un nouveau jeton de formulaire pour chaque essai afin de ne pas tester seulement le rejet CSRF.
5. Vérifier la connexion légitime après la période de verrouillage.

À dire si l'effet est observé

« Ces essais montrent le comportement de cette série de requêtes. Le succès avec le mot de passe connu sert de contrôle ; il ne constitue pas une découverte de mot de passe inconnu. »

Comparaison et contre-mesures — à dire

« La limitation des tentatives doit combiner état côté serveur et politique adaptée au risque. Il faut aussi tester les conséquences du verrouillage sur un utilisateur légitime. »

Limites et remise en état

Ne pas annoncer un débit ou une résistance à la force brute à partir de trois essais. Ne pas lancer de grande liste pendant la vidéo ; documenter séparément toute mesure plus longue.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

13. Identifiants de session faibles

Module : /vulnerabilities/weak_id/ • Sources : upstream/vulnerabilities/weak_id/source/

À dire avant le test

« Le module produit un cookie de démonstration appelé dvwaSession. Je vérifie si sa valeur est prévisible. »

À filmer et à faire

1. Ouvrir les outils réseau. Cliquer trois à cinq fois sur Generate.
2. Relever Set-Cookie et la valeur de dvwaSession ; au niveau low, rechercher la progression du compteur.
3. Prévoir la valeur suivante puis la comparer à la génération réelle.
4. Aux niveaux medium et high, comparer les sources à la suite observée ; un format haché ne suffit pas à garantir une source aléatoire.
5. Au niveau impossible, lire random_bytes(20) et observer l'en-tête de génération.

À dire si l'effet est observé

« La valeur suivante est prévisible dans cet échantillon et la source explique pourquoi. Ce cookie est le jeton pédagogique du module, pas une preuve de vol de la session PHP principale. »

Comparaison et contre-mesures — à dire

« La génération doit utiliser une source cryptographiquement sûre. Un échantillon visuellement irrégulier ne prouve pas une entropie suffisante ; je complète l'observation par l'analyse de la source. »

Limites et remise en état

Le niveau impossible ajoute Secure et utilise HTTP_HOST comme domaine de cookie. Sur notre installation HTTP avec port, inspecter les éventuels rejets du navigateur. Un cookie non enregistré n'est pas une preuve de sécurité cryptographique.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

14. CAPTCHA non sûr et saut d'étape

Module : /vulnerabilities/captcha/ • Sources : upstream/vulnerabilities/captcha/source/

À dire avant le test

« Je distingue la résolution du CAPTCHA de la validation serveur du processus. Une étape réussie doit être liée à une session et vérifiée lors de l'action finale. »

À filmer et à faire

1. Montrer que les clés reCAPTCHA ne sont pas configurées. Ne pas prétendre avoir validé le parcours externe.
2. Lire low.php : la branche step=2 ne refait pas la vérification CAPTCHA.
3. Pour tester uniquement cette branche, dans Repeater envoyer un POST authentifié vers /vulnerabilities/captcha/ avec Content-Type: application/x-www-form-urlencoded et corps :
step=2&password_new=Video-Lab-2026%21&password_conf=Video-Lab-2026%21&Change=Change
4. Si la branche est accessible, vérifier la modification par reconnexion puis restaurer le mot de passe. Si la page impose un prérequis bloquant, enregistrer le test comme non exécuté.
5. Lire impossible.php et préparer un rejeu avec configuration CAPTCHA valide ultérieurement.

À dire si l'effet est observé

« Le saut direct vers la seconde étape a permis la modification sans preuve serveur d'une première étape validée. Ce résultat concerne le processus de validation ; je n'ai pas cassé le service reCAPTCHA. »

Comparaison et contre-mesures — à dire

« Il faut contrôler la vérification au moment de la mutation et associer toute étape intermédiaire à un état serveur. Sans clés et connectivité adéquates, un échec du parcours corrigé ne permet pas de conclure à son fonctionnement normal. »

Limites et remise en état

La branche de succès est conditionnelle : les contraintes de configuration doivent être observées avant de la prononcer. Le parcours complet reste non validé hors ligne.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

15. Contournement de Content Security Policy

Module : /vulnerabilities/csp/ • Sources : upstream/vulnerabilities/csp/source/

À dire avant le test

« Une CSP définit quelles sources de scripts sont autorisées. Sa présence ne suffit pas : je vérifie si un attaquant peut utiliser une source ou un mécanisme autorisé. »

À filmer et à faire

1. Filmer Content-Security-Policy dans la réponse du niveau low ; expliquer les domaines autorisés. Les variantes utilisant des hébergeurs externes sont à préparer séparément.

2. Pour une démonstration locale reproductible, choisir medium et soumettre :

```
<script nonce="TmV2ZXIgzZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert('CSP_VIDEO')</script>
```

3. Filmer le nonce identique dans l'en-tête et dans la charge. Recharger et vérifier s'il reste constant.

4. Au niveau high, observer Solve et source/jsonp.php?callback=solveSum dans l'historique. Dans Repeater, tester callback=alert et inspecter la réponse comme code ; cette inspection seule ne prouve pas l'exécution navigateur.

5. Au niveau impossible, montrer le callback fixe et vérifier que Solve retourne toujours 15.

À dire si l'effet est observé

« Le script portant le nonce connu est accepté. Ce nonce statique ne constitue pas une autorisation imprévisible liée à une réponse particulière. »

Comparaison et contre-mesures — à dire

« Un nonce doit être imprévisible et renouvelé ; les sources de scripts autorisées doivent aussi être maîtrisées. Un endpoint JSONP permettant un callback arbitraire élargit cette surface. Je conserve un test fonctionnel du calcul pour la comparaison. »

Limites et remise en état

Le scénario medium ne dépend pas d'un hébergeur externe. Les scénarios low externes et l'exécution navigateur high doivent avoir leurs propres preuves. Ne pas attribuer à la seule CSP une absence de charge exploitable.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

16. Attaques JavaScript et confiance dans le client

Module : /vulnerabilities/javascript/ • Sources : upstream/vulnerabilities/javascript/source/

À dire avant le test

« Le module demande un mot et un jeton calculé par le navigateur. Je teste si un calcul entièrement disponible côté client peut servir de preuve d'autorisation. »

À filmer et à faire

1. Au niveau low, saisir success et soumettre : observer le comportement avant recalcul du jeton.

2. Dans la console de cette page locale, exécuter :

```
document.getElementById('phrase').value = 'success';  
generate_token();
```

3. Cliquer Submit et montrer le jeton envoyé. Lire la fonction MD5 de ROT13.

4. Pour medium et high, montrer la logique du code et utiliser les valeurs produites par fixtures/js-tokens.py dans Repeater avec phrase=success et token=la_valeur_du_niveau.

5. Ouvrir impossible : filmer l'explication de DVWA indiquant l'absence de ce niveau.

À dire si l'effet est observé

« J'ai pu recalculer un jeton accepté avec des informations disponibles côté client. L'obfuscation et le hachage d'un texte connu ne créent pas un secret. »

Comparaison et contre-mesures — à dire

« Les décisions d'autorisation et les secrets nécessaires doivent rester côté serveur. DVWA n'implémente pas de niveau impossible pour ce module ; il serait faux de présenter son écran comme un correctif testé. »

Limites et remise en état

Exécuter une commande manuelle dans la console prouve la contrôlabilité du client, pas une XSS distante. Le futur correctif métier doit définir précisément quelle autorisation le jeton est censé établir.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

17. Redirection HTTP ouverte

Module : /vulnerabilities/open_redirect/ • Sources : upstream/vulnerabilities/open_redirect/source/

À dire avant le test

« Je vérifie si le site permet de choisir librement une destination de redirection, ce qui peut détourner la confiance accordée à son adresse. »

À filmer et à faire

1. Cliquer un lien normal et repérer la route source/low.php et son paramètre redirect.

2. Ouvrir :

`http://127.0.0.1:4280/vulnerabilities/open_redirect/source/low.php?
redirect=http%3A%2F%2F127.0.0.1%3A4290%2Fdestination.html`

3. Filmer Location dans la réponse sans suivi automatique dans Repeater, puis la navigation vers la page témoin.

4. Tester la même destination sur source/impossible.php ; vérifier ensuite redirect=1 sur cette route.

À dire si l'effet est observé

« La réponse du serveur impose une destination que j'ai fournie et le navigateur la suit. La destination témoin appartient au laboratoire. »

Comparaison et contre-mesures — à dire

« La référence impossible associe des identifiants à des destinations définies côté serveur. La sélection d'un niveau ne neutralise pas nécessairement les fichiers source directement accessibles : je distingue le test de cette route de la sécurisation complète du déploiement. »

Limites et remise en état

Lancer le serveur témoin 4290 avant la scène. Ne pas confondre une redirection ouverte avec une SSRF : ici, c'est le navigateur qui suit la destination.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

18. Cryptographie : XOR, ECB, oracle et intégrité

Module : /vulnerabilities/cryptography/ • Sources : upstream/vulnerabilities/cryptography/source/

À dire avant le test

« Ce module contient plusieurs problèmes distincts. Je vais séparer la confidentialité, la manipulation de blocs et la vérification d'intégrité, plutôt que présenter un unique test de cryptographie. »

À filmer et à faire

A — Low : encoder hello, puis décoder la chaîne du défi. Exécuter fixtures/crypto-xor.py pour montrer le XOR répété et retrouver le message. Le mot Olifant appartient au mini-défi, pas au compte principal DVWA.

B — Medium : copier les trois jetons actuels Sooty, Sweep et Soo dans un fichier local. Les découper en blocs de 32 caractères hexadécimaux (16 octets). Composer bloc 1 de Sweep + bloc 2 de Soo + bloc 3 de Sooty + blocs restants de Sweep. Soumettre et observer identité, rôle et validité. Ne pas utiliser des jetons inventés ou d'une autre version.

C — High : présenter source/check_token_high.php et source/oracle_attack.php. Comparer un jeton valide et une altération en gardant un format valide ; distinguer erreur de format, padding et résultat métier. La récupération complète par oracle demande une répétition dédiée et des journaux ; ne pas la déclarer accomplie à partir de deux réponses.

D — Impossible : soumettre le jeton original, puis une modification d'un octet du ciphertext conservant un Base64 valide ; examiner le rejet d'intégrité, puis rejouer l'original.

À dire si l'effet est observé

« Pour chaque sous-test, je précise l'effet observé : récupération d'un message XOR, recomposition d'un jeton ECB, présence d'un signal d'oracle ou rejet d'une altération. Aucun de ces résultats ne signifie que l'algorithme AES lui-même a été cassé. »

Comparaison et contre-mesures — à dire

« Le code impossible emploie AES-256-GCM avec un tag d'authentification. Le nombre 256 désigne ici la taille de clé ; la taille de bloc AES reste 128 bits. CBC seul n'implique pas automatiquement un oracle : il faut une réaction exploitable à des ciphertexts choisis. La gestion de clé reste pédagogique, avec une constante dans le code. »

Limites et remise en état

Ne pas reprendre sans vérification les simplifications de l'aide DVWA. L'oracle complet est une scène à préparer, pas un résultat déjà acquis. Conserver quatre sous-fiches de preuve pour ce module.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

19. Sécurité des API : quatre sous-exercices

Module : /vulnerabilities/api/ • Sources : upstream/vulnerabilities/api/source/

À dire avant le test

« Les appels API constituent des points d'entrée indépendants de l'interface. Je compare versions, champs acceptés, appels système et mécanismes d'authentification. »

À filmer et à faire

A — Low, versions : dans Network, observer GET /vulnerabilities/api/v2/user/. Dans Repeater, remplacer v2 par v1 ; comparer les propriétés retournées, notamment password si présente. N'utiliser que les utilisateurs fictifs.

B — Medium, affectation de champs : capturer le PUT /vulnerabilities/api/v2/user/2 généré par le changement de nom. Ajouter level au JSON :

```
{"name":"morph","level":0}
```

Relire l'utilisateur avec GET ; vérifier si le rôle a changé. Sauvegarder puis restaurer sa valeur initiale.

C — High, commande : capturer le POST vers /vulnerabilities/api/v2/health/connectivity, avec Content-Type: application/json. Tester d'abord 127.0.0.1 puis :

```
{"target":"127.0.0.1; printf VIDEO_API_COMMAND"}
```

Observer le marqueur dans la réponse et relier à HealthController.php.

D — Impossible, authentification : utiliser les routes et paramètres du code LoginController.php. Capturer un login, l'obtention d'un access token, une requête autorisée et le renouvellement. Vérifier également une requête sans jeton. Lire les durées effectives dans Login.php avant de prévoir l'attente dans la vidéo.

À dire si l'effet est observé

« Je distingue ici une ancienne version trop bavarde, un champ privilégié accepté lors d'une mise à jour, une entrée interprétée par le shell et un parcours de jetons. Chaque sous-exercice nécessite sa propre preuve. »

Comparaison et contre-mesures — à dire

« Les réponses doivent exposer uniquement les propriétés nécessaires, les mises à jour accepter explicitement les champs autorisés, et les commandes éviter le shell. Le niveau impossible de ce module propose un exercice de type OAuth : il ne remplace pas les endpoints vulnérables et ne prouve pas leur correction. »

Limites et remise en état

Le ping API déjà testé prouve seulement la disponibilité. Les routes API ne suivent pas toutes le cookie de niveau. Ne pas dire que sélectionner impossible sécurise v1 ou connectivity. Présenter le flux d'authentification comme celui du laboratoire, pas comme un modèle de production.

Preuve à conserver : entrée, requête/réponse, effet et niveau. Si le résultat attendu n'apparaît pas, utiliser la phrase d'incident en fin de document ; ne pas lire la phrase de succès.

Parcours systématique des niveaux intermédiaires

Pour chacun des modules, ajouter une prise medium et high à la fiche si l'on annonce une étude de tous les niveaux. Montrer le code réellement chargé, rejouer le test low, identifier la protection ajoutée et concevoir un test ciblant sa limite. Un rejet de la charge low n'implique pas l'absence d'autres charges.

À dire : « Le premier essai échoue sur ce niveau. Je regarde maintenant ce qui a changé dans le code pour déterminer si la cause est une correction de fond, un filtrage partiel ou une différence de protocole. »

La cryptographie et l'API ont des objectifs distincts selon les niveaux. JavaScript n'a pas de correctif impossible. Les routes source directement exposées exigent une analyse indépendante du menu.

Table de suivi à recopier pour CHAQUE niveau et sous-scénario : identifiant du test ; compte ; entrée normale ; entrée de test ; effet attendu ; effet observé ; protection ; usage normal après test ; preuve ; statut (réussi / bloqué / non exécuté / à reprendre).

Présenter les correctifs personnels — scène future

Cette scène ne doit être annoncée comme terminée qu'après implémentation et validation.

À filmer : un diff Git limité au correctif, les requêtes comparables et le contrôle fonctionnel. Distinguer explicitement la source DVWA de notre modification.

À dire avant réalisation : « La comparaison précédente utilise les protections existantes de DVWA. La prochaine phase consiste à développer nos propres correctifs et à vérifier leur efficacité par rejeu. »

À dire après réalisation uniquement : « Dans ce commit [identifiant], j'ai modifié [fonction] pour [objectif]. Le scénario [identifiant] provoquait [effet] sur la référence. Après modification, je vérifie [état réel] et je montre que [usage légitime] reste fonctionnel. Cette conclusion est limitée aux cas testés. »

Ne pas simplement copier impossible.php et annoncer un travail original. Si du code upstream est repris, l'indiquer et expliquer l'adaptation.

Que dire si une démonstration échoue ?

« Le résultat attendu n'apparaît pas dans cette prise. Je vérifie le niveau, la session, la requête envoyée et les prérequis. À ce stade, je ne peux pas conclure que la faille est absente. »

Si redirection login : « La session n'est plus valide. Ce résultat concerne l'authentification de mon test ; je dois rétablir les préconditions avant de tester la vulnérabilité. »

Si CAPTCHA ou dépendance réseau : « Cette partie dépend d'un service qui n'est pas disponible dans notre configuration isolée. Je présente l'analyse du code et je marque la validation dynamique comme non exécutée. »

Si protection observée : « L'effet recherché n'est pas observé sur ce cas précis. Je complète le résultat par le code et par un usage normal, afin de distinguer un blocage de sécurité d'une panne. »

Si le site est lent : « Je conserve la durée réelle dans les preuves. Une accélération au montage sera indiquée et ne servira pas à une conclusion temporelle. »

Conclusion de la vidéo — version honnête au stade actuel

« Cette étude présente les mécanismes d'attaque sur les dix-neuf modules de la version DVWA retenue. Les scènes enregistrées et la matrice indiquent lesquelles ont été exécutées, lesquelles sont seulement analysées et lesquelles restent dépendantes d'une configuration supplémentaire.

Les résultats montrent pourquoi la sécurité doit porter sur la requête SQL, les contextes d'affichage, les autorisations serveur, les traitements de fichiers et les API. Une interface restrictive ou un test bloqué ne suffit pas à établir une protection générale.

Les protections intégrées à DVWA constituent des références pédagogiques. Leurs noms de niveaux ne sont pas des certifications, et certains modules changent d'objectif entre niveaux.

La contribution personnelle sera matérialisée par des correctifs identifiés dans Git et des tests de rejeu accompagnés de contrôles fonctionnels. Je ne présente pas cette phase comme achevée tant que ses preuves ne sont pas disponibles.

Les limites de ce travail sont celles d'un laboratoire intentionnellement vulnérable, d'un ensemble fini de scénarios et de dépendances externes partiellement indisponibles. Merci pour votre attention. »

Si les correctifs ont ensuite été réalisés, remplacer le quatrième paragraphe par les commits et résultats mesurés, avec les nombres réels. Ne pas conserver le futur par erreur.

Questions probables du jury et réponses préparées

Pourquoi DVWA ? — « Le code est accessible et les scénarios reproductibles. Cela facilite l'explication des causes et la comparaison. Je reconnais que le contexte métier est simplifié. »

Quelle est votre contribution ? — « Le protocole, les preuves, l'analyse critique et, après réalisation, les correctifs et tests identifiés. DVWA et ses protections préexistantes sont attribués à leurs auteurs. »

Toutes les attaques ont-elles été testées ? — « Tous les modules sont au périmètre. Voici la matrice distinguant les sous-tests exécutés des analyses et des cas bloqués. »

Pourquoi pas seulement un scanner ? — « Un scanner fournit des pistes. Je vérifie les effets, les autorisations et les usages légitimes ; les scénarios navigateur et métier demandent des contrôles spécifiques. »

Low bloqué sur high signifie-t-il sécurisé ? — « Non. La charge testée peut être filtrée sans que la cause soit supprimée. Je dois analyser la modification et tester ses limites. »

Impossible est-il invulnérable ? — « Non. C'est un nom pédagogique. Dans JavaScript, ce niveau n'existe pas réellement, et dans l'API il correspond à un autre exercice. »

Une XSS permet-elle toujours de voler les cookies ? — « Non. L'effet dépend du contexte et des protections. Je démontre ici une exécution de script sans annoncer une conséquence que je n'ai pas testée. »

Pourquoi un 200 ne suffit-il pas ? — « La réponse peut contenir un refus métier. À l'inverse, une erreur peut suivre une mutation déjà effectuée. Je vérifie l'état. »

Pourquoi ne pas publier DVWA sur Internet ? — « Le périmètre de ce travail est local ; les vulnérabilités intentionnelles ne sont utiles qu'à nos expériences contrôlées. »

Vérifications avant publication de la vidéo

Vérifier que chaque réussite annoncée possède une preuve visible ; que les niveaux et les comptes sont identifiables ; que les coupures et accélérations sont indiquées ; que les secrets montrés sont uniquement ceux du laboratoire ; que les tests non exécutés sont explicitement marqués ; que les auteurs de DVWA sont cités ; que la conclusion correspond à l'état réel du travail.

Ne pas afficher un tableau « 19/19 corrigés » si l'on a seulement ouvert 19 pages. Ne pas utiliser le niveau impossible comme une preuve de nos propres correctifs. Ne pas confondre la démonstration d'un mécanisme et la couverture de toutes ses variantes.

Sources et traçabilité

Dépôt officiel : <https://github.com/digininja/DVWA/tree/b496a5d3de6b967410155e1b7d3e51e9d035eb22>

Sources examinées : upstream/vulnerabilities/<module>/source/, index.php, fichiers help/help.php ; pour API, src/ et openapi.yml ; pour les accès, get_user_data.php et dvwa/includes/dvwaPage.inc.php.

Attention : l'aide peut simplifier ou diverger du code. Ce document tient notamment compte du cookie BAC, de l'absence de niveau JavaScript impossible et des sous-exercices distincts de l'API. Les résultats dynamiques de tournage restent à confirmer.

Documents du projet : docs/inventory.md, docs/images.json, VERSION, evidence/http-check.json et evidence/api-check.json.